

Testplan

Grupp 13

2026-03-30

Version 0.2



Status

Granskad	Szilvia Varro-Gyapay	2026-xx-xx
Godkänd	Szilvia Varro-Gyapay	2026-xx-xx

Projektidentitet

Grupp E-post: tddd96_2026vt_ax_13@groups.liu.se

Beställare: Johan Munck, Börjes Koncernen
Tfn: +46733318552
E-post: johan@borjeskoncernen.se

Handledare: Szilvia Varro-Gyapay
Tfn: +4613284092
E-post: szilvia.varro-gyapay@liu.se

Kursansvarig: Lena Buffoni
Tfn: +4613284046
E-post: lena.buffoni@liu.se

Projektdeltagare

Namn	Ansvar	E-post
Karl Lennartsson	Teamledare	karle221@student.liu.se
Simon Andersson	Dokumentansvarig	siman020@student.liu.se
Elias Facq	Analysansvarig	eliwa076@student.liu.se
Jacob Ahling Hult	Utvecklingsledare	jacah067@student.liu.se
Jakob Renqvist	Arkitekt	jakre732@student.liu.se
Viktor Wallberg	Kvalitetssamordnare	vikwa448@student.liu.se
Felix Hallström	Testledare	felha291@student.liu.se

INNEHÅLL

1	Inledning	1
1.1	Beställare	1
1.2	Översiktlig beskrivning av projektet	1
1.3	Syfte med testplanen	1
1.4	Dokumentstruktur och standard	1
2	Testorganisation	1
2.1	Testorganisation och ansvar	1
3	Testnivåer och testfall	1
3.1	Enhetstester	1
3.2	Integrationstester	2
3.3	Systemtester	2
3.4	Användbarhetstester	3
4	Planering och uppföljning	4
4.1	Tidsplan för testning	4
4.2	Risker kopplade till testning	4
4.3	Testrapportering	5

DOKUMENTHISTORIK

Version	Datum	Utförda förändringar	Utförda av		Granskad
0.1	2026-02-18	Första utkast	Felix ström	Hall-	2026-02-18
0.1	2026-02-20	Inlämning 1	Felix ström	Hall-	2026-03-20
0.2	2026-03-26	Inlämning 2	Felix ström	Hall-	2026-03-27

1 INLEDNING

1.1 Beställare

Beställaren i detta projekt är Johan Munck från Börjes Koncernen.

1.2 Översiktlig beskrivning av projektet

Projektets syfte är att bygga ett system som trafikledare ska använda för att tidigare kunna bedöma lönsamheten av en transportrutt. Systemet ska vara lättanvänt och ha ett intuitivt användargränssnitt. För mer information om projektets bakgrund och övergripande planering se projektplanen [1].

1.3 Syfte med testplanen

Syftet med denna testplan är att beskriva hur testning av systemet ska planeras och genomföras. Detta görs för att säkerställa att mjukvaran som utvecklas blir tillräckligt stabil och robust för att möta kundens krav och behov. Testplanen fokuserar på den testning som genomförs under första halvan av läsperioden och är ett levande dokument som kommer att uppdateras kontinuerligt under projektets gång.

1.4 Dokumentstruktur och standard

Denna testplan är utformad med utgångspunkt i IEEE Std 829-2008, som beskriver struktur och innehåll för testdokumentation inom programvaruutveckling [2]. Standarden har anpassats till projektets omfattning och arbetssätt.

2 TESTORGANISATION

2.1 Testorganisation och ansvar

Testning är ett gemensamt ansvar för hela projektgruppen. Varje utvecklare ansvarar för att testa sin egen kod genom enhetstester, medan övergripande planering, samordning och uppföljning av testningen sker gemensamt inom gruppen.

3 TESTNIVÅER OCH TESTFALL

3.1 Enhetstester

Enhetstester används för att verifiera enskilda funktioner och moduler, främst i backend.

Test-ID	Testnamn	Beskrivning
U1	Validering av indata	Kontrollerar att inmatningsfält valideras (ej tomma/ogiltiga)
U2	Test av intäktsmodul	Verifierar att systemets intäktsberäkning fungerar korrekt
U3	Hantering av saknad data	Säkerställer att backend hanterar fall där ruttdata saknas
U4	Felhantering i backend	Kontrollerar att fel i datahämtning/beräkning ger kontrollerat svar
U5	Prestanda för beräkning	Verifierar att isolerad beräkning sker inom rimlig tid
U6	Test av kostnadsmodul	Verifierar att systemets kostnadsberäkning fungerar korrekt
U7	Beräkning av körda kilometer	Kontrollerar att systemet beräknar avstånd korrekt

Tabell 1: Planerade enhetstestfall

3.2 Integrationstester

Integrationstester används för att verifiera att systemets olika komponenter fungerar korrekt tillsammans, särskilt kommunikationen mellan frontend, backend och externa system.

Test-ID	Testnamn	Beskrivning
I1	Frontend–Backend-kommunikation	Verifierar att frontend kan anropa backendens API och få svar
I2	HTTPS-kommunikation	Kontrollerar att frontend–backend kommunicerar via HTTPS
I3	JSON-responsformat	Säkerställer att backend returnerar resultat i JSON-format enligt specifikation
I4	Integration med kundens API	Verifierar att backend hämtar nödvändig ruttdata via kundens API
I5	Integration med inloggningssystem	Verifierar autentisering och att obehöriga anrop nekas
I6	Kryptering av lösenord	Verifierar att lösenord sparas krypterat i databasen

Tabell 2: Planerade integrationstestfall

3.3 Systemtester

Systemtester verifierar att hela systemet fungerar enligt kraven när det används via en webbläsare ur användarens perspektiv.

Test-ID	Testnamn	Beskrivning
S1	Inloggning i systemet	Verifierar att användare/admin kan logga in och komma åt systemet
S2	Inmatning av rutt	Kontrollerar att inmatningsdatan kan anges i gränssnittet
S3	Start av beräkning	Verifierar att beräkning kan startas via knapp och att anrop skickas
S4	Presentation av resultat	Kontrollerar att resultat visas korrekt och överskådligt
S5	Validering och felmeddelande	Säkerställer att tydligt felmeddelande visas vid saknad inmatning
S6	Meddelande vid saknad data	Verifierar att systemet visar meddelande när data saknas för angiven rutt
S7	Flera beräkningar utan omladdning	Verifierar att flera beräkningar kan göras utan sidladdning
S8	Stöd för olika webbläsare	Kontrollerar funktion i Chrome, Firefox och Edge
S9	Admin hanterar konton	Verifierar att administratör kan skapa och ändra användarkonton via gränssnittet
S10	Responsivitet	Kontrollerar att gränssnittet anpassar sig och fungerar på både dator och mobil
S11	Samtidiga användare	Verifierar att systemet hanterar flera samtidiga användare utan att krascha
S12	End-to-end svarstid	Mäter att en komplett lönsamhetsberäkning tar högst 1 sekund

Tabell 3: Planerade systemtestfall

3.4 Användbarhetstester

Användbarhetstester används för att utvärdera hur lätt systemet är att förstå och använda för slutanvändare. Testerna genomförs manuellt och är delvis explorativa.

Test-ID	Testnamn	Beskrivning
A1	Första användning utan instruktion	Utvärdera om en testperson förstår hur inloggning och ruttinmatning sker utan hjälp
A2	Tydlighet i resultatpresentation	Utvärderar om resultatet är lätt att tolka och förstå
A3	Begriplighet i felmeddelanden	Utvärderar om felmeddelanden är tydliga och hjälper användaren vidare
A4	System Usability Scale (SUS)	Låter testpersoner fylla i SUS-enkät efter användning för att verifiera poäng över 70

Tabell 4: Planerade användbarhetstestfall

4 PLANERING OCH UPPFÖLJNING

4.1 Tidsplan för testning

Testning planeras i grunden att ske löpande under utvecklingsarbetet i takt med att funktionalitet färdigställs. Enhetstester skrivs och körs kontinuerligt av respektive utvecklare i samband med att ny kod implementeras, medan integrationstester och systemtester genomförs när en sammanhängande del av funktionaliteten är på plats.

Läsperiod 1 (Första halvan av projektet): Under projektets inledande fas låg huvudfokus på att etablera systemets arkitektur och bygga upp den grundläggande funktionaliteten. Den övergripande planen för denna period var att testa moduler allteftersom de blev klara, men formella och storskaliga testomgångar var begränsade i väntan på att systemets delar skulle integreras. Utvecklarna ansvarade dock för att testa sin egen kod (enhetstester) under arbetets gång.

Läsperiod 2 (Andra halvan av projektet): Under projektets andra halva övergår testningen till en mer systematisk och iterativ fas. Planen för period 2 är att införa veckovisa testomgångar av den funktionalitet som är tillräckligt klar. Inför varje veckomöte genomförs relevanta testfall för att verifiera nya funktioner och säkerställa att ny kod inte har påverkat befintlig funktionalitet.

Resultaten från dessa veckovisa tester ligger sedan till grund för den formella testrapporteringen (se avsnitt 4.3) och hjälper gruppen att prioritera nästkommande veckas arbete.

Om ett testfall inte går igenom åtgärdas fel i första hand direkt av ansvarig utvecklare. Vid större fel, eller fel som påverkar flera delar av systemet, dokumenteras felet och följs upp tills dess att en åtgärd har verifierats genom omkörning av relevanta testfall. Berörda tester körs alltid om vid betydande uppdateringar för att förhindra att befintlig funktionalitet förstörs.

4.2 Risker kopplade till testning

En risk med testningen är ren tidsbrist, vilket kan leda till att vissa tester inte genomförs i önskad omfattning eller att testning prioriteras ned vid hög arbetsbelastning. En annan risk är otydliga eller föränderliga krav, vilket kan medföra att testfall behöver revideras under projektets gång. Ytterligare en teknisk risk är bristande testdata eller begränsad tillgång till externa system, såsom kundens API eller inloggningssystem, vilket kan försvåra integrationstester.

Utöver de tekniska och resursmässiga riskerna identifieras även en kognitiv risk i form av bekräftelsebias (*confirmation bias*). Forskning visar att när utvecklare testar sin egen kod under tidspress, ökar risken för att testerna primärt utformas för att bekräfta systemets huvudflöden, medan oväntade fel och kantfall förbises [3]. Då projektet styrs av en fast tidsbudget hanteras denna risk genom att testningen aldrig uteslutande förlitar sig på enhetstester utförda av kodens skapare.

Samtliga av dessa risker, både de tekniska och kognitiva, hanteras genom kontinuerlig uppföljning av testresultat, kompletterande integrationstester utförda av andra gruppmedlemmar, explorativa tester med kunden, samt genom att relevanta tester alltid körs om vid större förändringar för att förhindra regressionsfel.

4.3 Testrapportering

För att säkerställa spårbarhet och kontinuerlig kvalitetskontroll sammanställs en formell testrapport varje vecka i anslutning till projektets statusrapportering. Denna process utgår från riktlinjerna i IEEE Std 829-2008 [2] och syftar till att ge en tydlig överblick över systemets stabilitet när nya framsteg har skett.

Den veckovisa testrapporten innehåller:

- En sammanfattad bedömning av produktens aktuella teststatus.
- En testlogg över de testfall som körts under veckan och deras utfall (godkänd/underkänd).
- Insamlade mätdata för att följa upp kvaliteten över tid.
- En incidentrapport över identifierade fel (buggar).

Genom att integrera testrapporteringen i den veckovisa uppföljningen säkerställs att eventuella fel upptäcks och hanteras tidigt i utvecklingsprocessen, samt att beställare och handledare får kontinuerlig insyn i produktens kvalitet.

REFERENSER

- [1] Grupp 13, "Projektplan för kandidatprojekt tddd96," Linköpings Universitet, Tech. Rep., 2026.
- [2] "Ieee standard for software and system test documentation," IEEE, Tech. Rep. IEEE Std 829-2008, 2008. [Online]. Available: <https://doi.org/10.1109/IEEESTD.2008.4578383>
- [3] I. Salman, B. Turhan, R. Ramač, and V. Mandić, "Confirmation bias and time pressure: A family of experiments in software testing," *IEEE Transactions on Software Engineering*, vol. 49, no. 12, pp. 5203–5222, 2023.